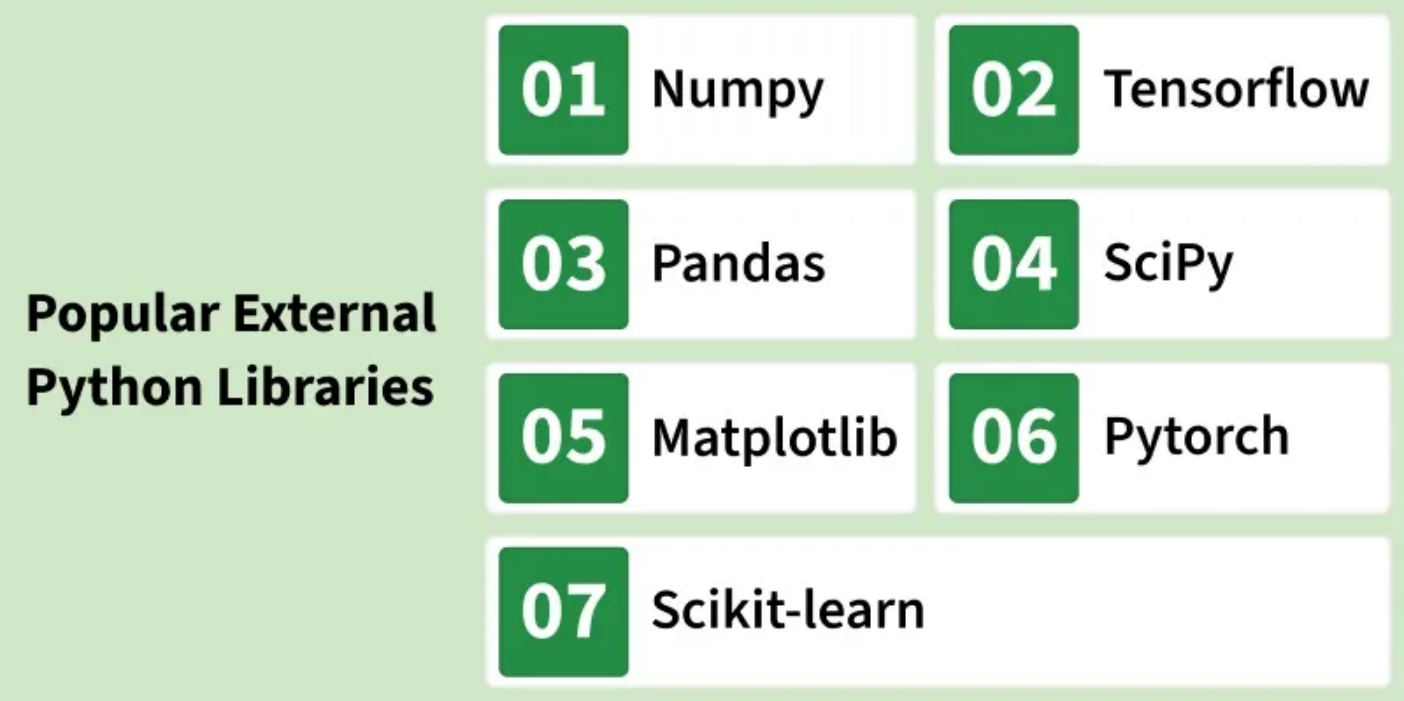
Libraries in Python

Last Updated : 13 Nov, 2025

In Python, a library is a group of modules that contain functions, classes and methods to perform common tasks like data manipulation, math operations, web scraping and more. Python libraries make coding faster, cleaner and more efficient by providing ready-to-use solutions for different domains such as data science, web development, machine learning and automation.



Popular External Python Libraries

Working of Python Library

When you import a library in Python, it gives access to pre-written code stored in separate modules. In simple terms instead of writing the logic for a task, you import the library that already has it.

For example, on Windows, libraries are stored as .dll (Dynamic Link Libraries) and on Linux/macOS as .so files. When you run your code, Python automatically loads these modules and makes their functions available to use.

Types of Python Library

Python libraries are divided into two main types:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

1. Built-in Python Standard Library

It is a collection of modules that come bundled with every Python installation, we don't need to install anything separately. Most of these modules are written in C for better performance.

Examples of built-in modules:

- **math:** Mathematical operations
- **os:** Interact with the operating system
- **datetime:** Date and time operations
- **random:** Generate random numbers
- **json:** Handles JSON data encoding and decoding.

2. External Python Libraries

External (third-party) libraries are not included with Python by default. You can install them easily using the pip package manager. Popular External Python Libraries:

1. **NumPy:** It, short for Numerical Python, is the core library for numerical and scientific computing in Python. It provides powerful tools for creating and manipulating arrays, matrices and multidimensional data.
2. **Pandas:** It is a data analysis and manipulation library built on top of NumPy. It introduces data structures like DataFrame and Series that make it easy to handle structured data efficiently.
3. **Matplotlib:** It is a data visualization library that helps you create a wide variety of static, animated and interactive plots. It supports charts such as bar graphs, line charts, histograms and scatter plots.
4. **SciPy:** It, short for Scientific Python, extends the functionality of NumPy by providing tools for advanced mathematical, scientific and engineering computations. It includes modules for optimization, integration, signal processing and linear algebra.
5. **TensorFlow:** It is an open-source machine learning and deep learning framework developed by Google. It allows developers to build and train neural

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

networks for tasks such as image recognition, natural language processing and predictive modeling.

6. **Scikit-learn:** It is a popular machine learning library built on top of NumPy and SciPy. It offers simple and efficient tools for classification, regression, clustering and dimensionality reduction.
7. **Scrapy:** It is a web scraping and data extraction library designed to efficiently crawl websites and gather structured information. It allows developers to create spiders that automatically navigate web pages and collect data.
8. **PyTorch:** It, developed by Facebook's AI Research Lab, is a deep learning framework that provides dynamic computation graphs and easy debugging. It supports GPU acceleration, making it suitable for high-performance training of neural networks.
9. **PyGame:** It is a cross-platform library used for developing 2D games and multimedia applications. It provides modules for handling graphics, sound and user input with ease.
10. **PyBrain:** It, short for Python-Based Reinforcement Learning, Artificial Intelligence and Neural Network, is a beginner-friendly machine learning library. It offers pre-built algorithms and flexible tools for training neural networks and reinforcement learning models.

Using Libraries in Python Programs

To use any library, it first needs to be imported into your Python program using the import statement. Once imported, you can directly call the functions or methods defined inside that library. You can import libraries in three main ways:

1. **Import the entire library:** import library_name for example, import math
2. **Import a specific function or class:** from library_name import function_name for example, from math import sqrt
3. **Import a library with an alias:** import library_name as alias for example, import pandas as pd

Example 1: This program imports the entire math library and uses one of its functions.

```
import math
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
A = 16
print(math.sqrt(A))
```

Output

```
4.0
```

Explanation:

- Here, the complete math library is imported and we use math.sqrt() to calculate the square root of 16.
- Since the full library is imported, we must prefix the function with the library name (math.).

Example 2: This program imports only selected functions from an external library to simplify usage.

```
from numpy import array, mean
a = array([10, 20, 30, 40, 50])
print(mean(a))
```

Output

```
30.0
```

Explanation:

- Here, only the array() and mean() functions are imported from the NumPy library.
- array() is used to create a NumPy array from a list and mean() calculates the average value of all elements in the array.
- Since these functions are imported directly, we don't need to use the numpy. prefix each time we call them.

Article Tags: Python

Explore

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Python Fundamentals

Python Data Structures

Advanced Python

Data Science with Python

Web Development with Python

Python Practice

Do Not Sell or Share My Personal Information

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF